Agentic Code Workbench

英語版

自然言語で記述した解析カードから LHCO 解析用 Python コードを生成し、実行と意味内容の確認を行うためのワークベンチです。入力仕様の曖昧性を確認する Oracle、Helper Selector、Code Generator、実行・修復処理、Tracer、Critique を含みます。

LHCO 固有のプロンプト、helper registry、解析規則は `domains/lhco-domain/` にまとめています。生成・選択・レビューの共通部品はそれぞれ独立しています。

実行環境の準備

Python 3.10 以上、Bash、および以下の外部パッケージを使用します。コマンドは、展開したパッケージのルートディレクトリから実行してください。

```
python -m pip install 'huggingface-hub>=0.27' 'requests>=2.31' numpy
matplotlib
```

パッケージ	用途
`huggingface-hub`	Helper Selector、Code Generator、Oracle の API 接続
`requests`	Tracer/Critique の API 接続
`numpy`、`matplotlib`	生成解析コードの数値処理と図出力

必要なら、先に `python3 -m venv .venv` と `source .venv/bin/activate` で専用の環境を用意してください。上記は必要な依存関係の指定であり、特定バージョン一式の固定ではありません。実験条件を記録する場合は、動作確認した環境の `python --version` と `python -m pip freeze` を保存してください。

モデルへの接続は Hugging Face API を使用します。利用するモデルへアクセスできるトークンを設定してください。モデルの利用には API の利用条件・料金が適用されます。

```
export HF_TOKEN="your-token"
```

本パッケージには、ローカル Transformers 推論を実行する runner は含まれていません。以下の shell スクリプトは必要な `src/` を参照パスに追加するため、各部品の editable install は不要です。

1ケースを実行する

Case 1 を同梱データで実行する例です。

```
bash scripts/run_qwen3_14b.sh \
  examples/cases/user_input_1.txt \
  data/signal_1.lhco \
  runs/case1
```

引数は次の3個です。

```
bash scripts/run_qwen3_14b.sh CARD LHCO_FILE RUN_DIR
```

引数	内容
`CARD`	解析要求を記述した入力カード
`LHCO_FILE`	生成コードへ渡す LHCO データ
`RUN_DIR`	カード、生成コード、実行記録、レビュー結果の出力先

処理は次の順序で進みます。

1. Helper Selector が registry から helper を選び、依存関係を展開します。
2. `[ADDITIONAL_HELPERS]` を付けた生成用カードを作ります。
3. Code Generator が Python コードを生成します。必要なら1回の継続生成を行います。
4. コードを実行します。実行に失敗した場合は修復を試み、それでも失敗すればカードから再生成します。
5. Tracer が実際の実装内容を抽出し、Critique がカードとの対応を確認します。
6. Oracle が入力カードの曖昧性を事後確認し、次の見直しに使える報告を保存します。

`[ADDITIONAL_HELPERS]` は生成時の推奨一覧であり、使用できる関数をその一覧だけに限定するものではありません。

既定のモデルは `Qwen/Qwen3-14B`、修復・再生成の試行数 `MAX_RETRIES` は3です。各試行では修復後に実行し、失敗した場合に再生成後の実行を行います。各試行の記録は個別に保存します。

解析実行が成功しなくても、生成コードが得られれば Tracer/Critique の対象になります。実行成功と、レビュー処理の完了は別に確認してください。

モデルを変更する

スクリプトを複製せず、`MODEL` を指定して変更できます。

```
MODEL=Qwen/Qwen3-32B bash scripts/run_qwen3_14b.sh \
  examples/cases/user_input_2.txt \
  data/signal_2.lhco \
  runs/case2_32b
```

この指定は Helper Selector、Code Generator、Fixer、Tracer、Critique、事後 Oracle に共通して適用されます。スクリプト名の `14b` は既定モデルを示します。

修復・再生成の試行数も変更できます。

```
MAX_RETRIES=2 bash scripts/run_qwen3_14b.sh \
  examples/cases/user_input_1.txt data/signal_1.lhco runs/case1_retry2
```

入力カードとデータ

カードは `[SELECTION_CUTS]`、`[PLOTS_FOR_VALIDATION]`、`[OUTPUT_STRUCTURE]` に解析要求を記述します。同梱の `examples/cases/` を入力例として利用できます。

Case	解析対象	入力カード	データ
1	W 解析	`user_input_1.txt`	`signal_1.lhco`
2	トップ解析	`user_input_2.txt`	`signal_2.lhco`
3	二光子解析	`user_input_3.txt`	`signal_3.lhco`
4	WZ・三レプトン解析	`user_input_4.txt`	`signal_4.lhco`
5	VBF H → ττ 解析	`user_input_5.txt`	`signal_5.lhco`
6	Case 2 の要求を明確化した解析	`user_input_6.txt`	`signal_6.lhco` または `signal_2.lhco`
7	Case 5 の要求を明確化した解析	`user_input_7.txt`	`signal_7.lhco` または `signal_5.lhco`

カードは `examples/cases/`、データは `data/` にあります。Case 6 と Case 2、Case 7 と Case 5 は、それぞれ同じプロセス・データを別のカードで解析します。`signal_6.lhco` と `signal_7.lhco` は、シンボリックリンクを使わず、それぞれ `signal_2.lhco` と `signal_5.lhco` のコピーを用意しています。

出力と結果の確認

出力先	内容
`cards/original.txt`	runner に渡したカード
`cards/accepted.txt`	主処理で受け付けたカード
`cards/generation.txt`	helper 選択を反映した生成用カード
`helpers/selection.json`、`summary.txt`、`raw_response.txt`	helper 選択結果とモデル応答
`generation/result.json`、`response_*.txt`	生成結果と初回生成の応答記録
`generation/src/analysis.py`	生成コード。実行回復を行う場合は試行に伴い更新

出力先	内容
`execution/generated_analysis.py`	最後に実行した解析コード
`execution/stdout.txt`、`cutflow.txt`	成功した実行の標準出力。`cutflow.txt` は互換用の同内容ファイル
`execution/` 内の図など	生成された解析コードが保存した出力
`execution/generation_info/`	初回実行、修復、再生成のコード、終了コード、標準出力、エラーなど
`critique/tracer/final_output.txt`	Tracer の実装記述
`critique/findings/response_001.txt`、`filtered_review.txt`	Critique の結果
`oracle/request.json`、`user_message.txt`、`response_001.txt`	事後 Oracle の設定、入力、報告

解析実行の成否は、cutflow、図、各試行のログ、および `generation_info/` の結果 JSON にある `returncode`・`succeeded` で確認できます。実行成功は、物理的な正しさやカードへの適合を保証するものではありません。Tracer/Critique の結果も含めて確認してください。

通常の runner は同じ出力ディレクトリを再利用できます。固定名のファイルは上書きされ、今回作成されなかった古い図や応答ファイルは残る場合があります。再利用時は今回の実行記録も確認してください。

全ケースの実行と時間計測

ルートにある `run_all_timed.sh` で Case 1–7 をまとめて実行できます。最初は各1回で確認します。

```
bash run_all_timed.sh timing_check 1
```

各10回、計70実行を行う例です。

```
bash run_all_timed.sh frozen_timing 10
```

`frozen_timing/run_01/case1/` のように結果を保存し、`frozen_timing/timing_cases1_7.tsv` に時刻、秒数、状態を記録します。Case 6・7 の実行には、それぞれ `signal_2.lhco`・`signal_5.lhco` を使います。

この計測用スクリプトは既存の出力先を拒否します。既に同名のディレクトリがある場合は新しい名前を指定してください。過去の実験を削除する必要はありません。

計測時間には主処理と事後 Oracle が含まれます。TSV の `success` は各 runner が正常終了したことを示し、生成解析コードの実行成功を保証しません。各 Case の実行記録と合わせて確認してください。一部の Case が失敗しても、スクリプトは後続 Case の実行を続けます。

全ケースのモデルを変える場合も同じ指定を使えます。

```
MODEL=Qwen/Qwen3-32B bash run_all_timed.sh timing_32b 1
```

Oracle を単独で使う

コード生成前に入力カードを見直したい場合は、Oracle を単独実行します。

```
bash scripts/run_oracle_lite.sh \
  examples/cases/user_input_2.txt runs/oracle_lite_case2
```

報告は `runs/oracle_lite_case2/response_001.txt` に保存されます。報告を人が確認し、必要に応じてカードを修正してから生成 runner に渡してください。Oracle の報告はカードへ自動適用しません。

入口	プロファイル	既定モデル
`scripts/run_oracle.sh`	`oracle`	`Qwen/Qwen3-14B`
`scripts/run_oracle_lite.sh`	`oracle-lite`	`Qwen/Qwen3-14B`
`scripts/run_oracle_lite_extensive.sh`	`oracle-lite-extensive`	`Qwen/Qwen3-14B`
`scripts/run_oraclet_8b.sh`	`oraclet`	`Qwen/Qwen3-8B`

`oracle` は共通の指示・出力規則とドメイン文脈を組み合わせます。`oracle-lite` と `oracle-lite-extensive` は単一ファイルのプロンプトを使い、`oraclet` はドメイン側の完全なプロンプトを使います。`lite` は使用モデルの大きさを表す名称ではありません。

`oracle-lite` は候補を絞り込み、重要な未確定事項を最大3件報告します。`oracle-lite-extensive` は網羅性を優先し、件数を制限せず候補を報告するため、出力を後で検討する必要があります。

モデル、実行回数、出力トークン上限、温度を変更する例です。

```
MODEL=Qwen/Qwen3-32B RUNS=3 MAX_TOKENS=1800 TEMPERATURE=0.2 \
bash scripts/run_oracle_lite.sh \
  examples/cases/user_input_2.txt runs/oracle_lite_case2_32b
```

一般 runner で同等の指定をする場合は、`PROFILE=oracle-lite ORACLE_OUTPUT_CONTRACT=none` を指定します。`oracle-lite`、`oracle-lite-extensive`、`oraclet` には別の system output contract を追加しません。

主処理の後の Oracle は既定で `oracle` です。変更する場合は `run_qwen3_14b.sh` に `ORACLE_PROFILE` と `ORACLE_OUTPUT_CONTRACT` を渡します。

```
ORACLE_PROFILE=oracle-lite ORACLE_OUTPUT_CONTRACT=none \
bash scripts/run_qwen3_14b.sh \
  examples/cases/user_input_1.txt data/signal_1.lhco runs/case1_lite
```

旧 Oracle プロンプトは `packages/golden-axe-oracle/src/golden_axe/profiles/oldfiles/` に保存しています。旧版が必要な場合は、対応するファイルを現行プロンプトファイルへ差し替えて使用してください。

テスト

```
bash scripts/test_all.sh
```

各部品と統合処理をモックで検査します。モデルのダウンロードや API 接続は不要です。実モデルでの動作確認は、先に1ケースを実行し、その後 `run_all_timed.sh` で全ケース各1回へ進めてください。

追加機能：Tracer/Critique だけを再実行する

`scripts/run_qwen3_32b_postreview.sh` は、既存の解析コードを使い、Tracer/Critique だけを既定で32Bモデルに変更して再実行する追加実験用の機能です。この postreview 実験は論文には採用していません。コード生成と解析コードの実行はやり直しません。

```
bash scripts/run_qwen3_32b_postreview.sh 2 7
```

この例は `examples/cases/user_input_2.txt` と `frozen_timing/run_07/case2/execution/generated_analysis.py` を読み、同じ Case ディレクトリの `critique_qwen3_32b/` に保存します。このスクリプトは既存の出力先を拒否します。入力を別の場所へ移動している場合は、スクリプト内のパスを現在の配置に合わせてください。

任意の既存コードを直接指定したい場合は、次の形でも実行できます。

```
PYTHONPATH=packages/critique-pipeline/src \
python -m critique_pipeline.cli run \
  --card runs/case1/cards/generation.txt \
  --code runs/case1/execution/generated_analysis.py \
  --tracer-model Qwen/Qwen3-32B \
  --critique-model Qwen/Qwen3-32B \
  --domain-context domains/lhco-
domain/src/lhco_domain/prompts/critique_context.txt \
  --n-runs 1 --no-repair \
  --run-dir runs/case1/postreview_32b
```

追加評価：Tracer 出力の忠実性を評価する

`scripts/evaluate_tracer.py` は、入力カード、生成コード、Tracer 出力を読み、各要求項目について追跡の有無、ソース位置、依存関係、ハルシネーションを評価して CSV に保存します。解析コードの物理的正しさを評価する Critique とは別の処理です。

この機能だけは OpenAI API を使います。追加の依存と認証を設定してください。

```
python -m pip install openai
export OPENAI_API_KEY="your-key"
```

利用するモデル名を `EVALUATION_MODEL` に設定し、1件ずつ実行できます。スクリプトは Responses API と `reasoning.effort=low` を使うため、その指定に対応するモデルを使用してください。

```
python scripts/evaluate_tracer.py \
  --case 1 --run 1 \
  --task-card examples/cases/user_input_1.txt \
  --code runs/case1/execution/generated_analysis.py \
  --trace runs/case1/critique/tracer/final_output.txt \
  --model "${EVALUATION_MODEL:?Set EVALUATION_MODEL}" \
  --out tracer_evaluation_check/case1_run01.csv
```

`--case` と `--run` は CSV の識別情報で、入力パスは各引数で明示します。出力には要求項目ごとの評価と、コードに裏付けられない追加の Tracer 主張が含まれます。評価結果は人が確認して利用してください。

ディレクトリ構成

パス	役割
`packages/workbench-contracts/`	共通の型・成果物配置・プロトコル
`packages/helper-selector/`	helper 選択と依存関係の展開
`packages/golden-axe-oracle/`	入力仕様の曖昧性確認
`packages/critique-pipeline/`	Tracer と Critique
`backends/code-generator/`	汎用のコード生成・継続・実行・修復
`domains/lhco-domain/`	LHCO 固有のカード解釈・プロンプト・registry・規則
`domains/dummy-domain/`	ドメイン交換を確認する小さなテスト用の例
`workflows/`	accepted card 以降の処理の合成
`scripts/`	単独実行、Oracle、テスト、追加評価の入口
`examples/cases/`	解析カードの例
`data/`	LHCO 入力データ
`paper_data/`、`recalculation/`	論文の保存済み実験・再計算資料

パス	役割
`tests/`	統合テスト
`run_all_timed.sh`	全ケースの実行と時間計測
`README/`	以前の構成・開発履歴に関する文書

現行 registry は、選択済みのオブジェクト・コレクションを使って低水準 helper を組み合わせる方針です。イベント全体からオブジェクトを取り直す旧 helper の一部は、既定の選択対象から除いています。旧 registry は `registry_legacy.py`、旧生成プロンプトは `prompts/codegen_system_legacy.txt` に保存しています。

`paper_data/` と `recalculation/` は保存済みの論文資料です。通常のテストや新しい実験の開始時に削除・初期化する必要はありません。再実験は別の出力先に保存し、論文に用いた結果と区別してください。

以前の Streamlit GUI と MLP/GNN/Transformer アプリケーションは、このコード生成ワークベンチの対象に含まれません。